

EPOCHESQUE 2.0

These are TRACKS, not fixed problem statements. Each track gives you a theme and rules to build against. Inside a track, you choose what exact product or agent to build.

Track 1: THE GLASS BOX PROBLEM

Your AI pipeline works today. Nobody knows why it will break tomorrow, and when it does, nobody can tell what went wrong.

What this track is about

- Build an AI system (agent, RAG app, or LLM pipeline) that you can fully see inside every step, every decision, every cost.
- You must add observability: log and trace every LLM call, tool call, and step, so a judge can replay exactly what happened and why.
- You must also handle context engineering — keep the AI accurate and on topic even in long conversations, by summarizing, trimming, or selecting what information the model actually sees.

Rules

- The system must produce a trace/log for every run — not just a final answer. A judge should be able to open it and see step by step what happened.
- You must show at least one real failure case you caught using your own tracing (e.g. wrong tool call, hallucination, context overflow) and how you fixed or flagged it.
- Long or messy conversations must not break the system — test it with at least 15–20 back and forth turns.
- Track your token usage / cost per run and show it to the judges.
- Off the shelf observability tools are allowed, but you must explain what they are tracking and why, no black box usage.

Track 2: BUILD THE BRAIN, NOT THE PUPPET

Anyone can plug into a ready made agent framework and call it AI. Can you build the thing the framework is hiding from you?

What this track is about

- Build your own small agent framework from scratch — the core loop that lets an AI plan, decide, call tools, and act on the result.
- Then use YOUR framework to build a real agent that solves an actual task end to end (research, coding help, automation, whatever you choose).
- This track is about understanding how agents work under the hood, not stitching together someone else's library.

Rules

- The core agent loop (plan → act → observe → repeat) must be written by your team, not imported from LangChain/CrewAI/AutoGen/similar.
- You may use LangChain (or similar) ONLY for small utilities (e.g. calling an LLM API), never for the actual agent logic or decision loop.
- Your agent must use at least 2 different tools (e.g. search, calculator, file access, an API) and choose between them on its own, no hardcoded if/else routing to fake it.

- Show what happens when a tool call fails or returns something unexpected, your agent should notice and recover, not silently break.
- Bonus (not required): compare your framework's behavior on the same task against a standard framework like LangChain, and explain the difference.

Track 3: THE ASSISTANT THAT NEVER FORGETS... OR DOES IT?

A search engine finds facts. Your assistant needs to remember people, decisions, and changes over time, and know when something it "knew" is no longer true.

What this track is about

- Build an AI assistant or agent that remembers things ACROSS sessions, not just within one chat, but across days, users, or conversations.
- Use a vector database and RAG to store and retrieve information, but go further: design real memory — what to keep, what to update, what to forget.
- The hard part is not storing facts. It's handling facts that change, contradict each other, or go stale over time.

Rules

- Your system must persist memory between sessions — closing and reopening the app should not wipe what it learned.
 - It must correctly handle contradicting information given at different times (e.g. a user says X on day 1, then says the opposite on day 3). judges will test this live.
 - Plain "dump everything into a vector DB and retrieve top k" will not survive judging, you must show a clear memory design: what gets stored, what gets updated, what gets removed or decayed, and why.
 - You must be able to explain, for any answer your system gives, WHICH stored memory it used to get there.
 - Multi user memory (keeping different users' memories separate and private) is a strong plus, not mandatory.
-

Judging Matrix

Criteria	Weight	What judges are checking
Problem Depth	25%	Did you take on the easiest version that technically fits the track? Or is it a real world problem?
Technical Execution	25%	Does it actually work end to end? Is the build solid, or held together for the demo only?
Rule Compliance	20%	Did you follow the specific rules of your chosen track?

Criteria	Weight	What judges are checking
Edge Case Handling	15%	What happens when judges push it, bad input, long conversation, contradicting data, a failed tool call?
Explanation & Demo	15%	Can the team clearly explain what's happening under the hood, not just click buttons?